Proxmox VE Migration

Day 2: Importing VMs & Guest Adaptation

Day 2 — Agenda

1. Recap: Day 1 Key Points
2. Migration Options & Decision Criteria
3. Preparing the Proxmox Target
4. Proxmox VE Import Assistant for ESXi
5. Migrating Linux Test VMs
6. Migrating Windows Server VMs
7. Migrating FreeBSD Test VMs

8. Guest Adaptation — Drivers, Boot, Network, Storage
9. Network & Storage Follow-up
10. Function Tests for Migrated VMs
11. Troubleshooting Import & Boot Issues
12. Performance Baseline & Tuning
13. Lab: Migrate and Validate Three VMs

1. Recap: Day 1 Key Points

- Proxmox VE is a **distributed** Type 1 platform; config lives in `/etc/pve`
- Migrated VMware VMs **stay VMs**; containerization is a later project
- **Assessment, wave planning, and rollback** determine migration success
- Networking (bridges/VLANs) and storage backends must be **designed first**
- Keep source VMs untouched until acceptance criteria are met

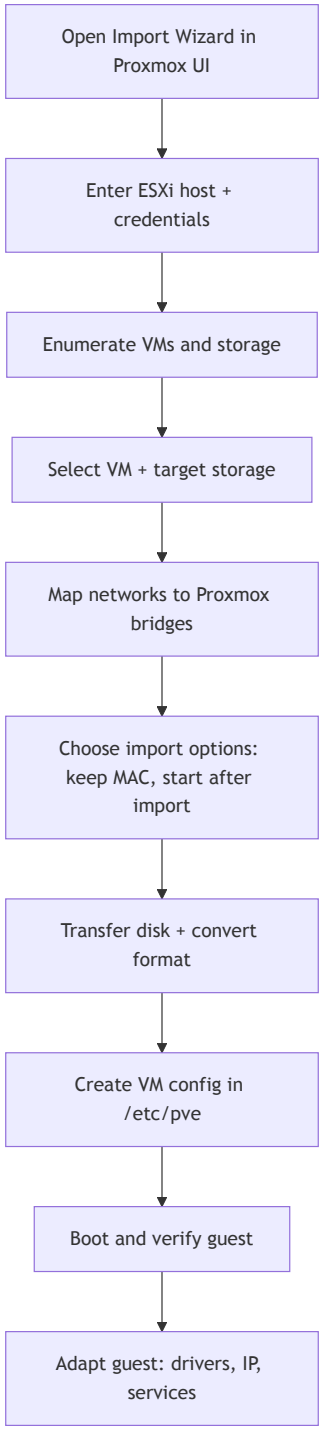
Today we execute: import real VMs, adapt their guests, and prove they work.

2. Migration Options & Decision Criteria

Option	Downtime	When to use	Notes
Cold import (recommended)	VM downtime during copy	Most migrations	Power off source VM, import disk, validate, cut over
Snapshot-based import	Shorter, but consistency risk	Databases/apps needing minimal downtime	Quiesce app, snapshot, import, reconcile deltas
Live migration (Proxmox only)	Seconds	Moving already-migrated VMs between Proxmox nodes	Requires shared storage or storage migration

Option	Downtime	When to use	Notes
Storage migration	Minutes to hours	Relocating disks between Proxmox storage	`qm migrate <vmid> <node> --with-local-disks`
Cross-cluster migration (PVE 8+)	Variable	Moving VMs between two Proxmox clusters	Remote migration API; useful for phased datacenter moves
Replication + cutover	Very short	DR-style move	ZFS replication or PBS restore to target, then final sync

Important: the ESXi import path is a **cold migration**. The source VM must be powered off (or you accept an inconsistent, crash-consistent copy). For databases, plan an application-level export/import or a final delta sync.


Open Import Wizard in Proxmox UI
Enter ESXi host + credentials
Enumerate VMs and storage
Select VM + target storage
Map networks to Proxmox bridges
Choose import options: keep MAC, start after import
Transfer disk + convert format
Create VM config in /etc/pve
Boot and verify guest
Adapt guest: drivers, IP, services

3. Preparing the Proxmox Target

Before importing, make the target boring and predictable.

```
# 1. Verify node health and version
pveversion -v
# 2. Repositories
cat /etc/apt/sources.list.d/*.list
apt update
# 3. Time sync (certificates, corosync, logs)
timedatectl status
# 4. Storage
pvesm status
pvesm add zfspool local-zfs --pool rpool/data --content images,rootdir
# 5. Network
cat /etc/network/interfaces
bridge vlan show 2>/dev/null || true
# 6. User/ACL for migration operators
pveum user add migrator@pve
pveum acl modify / --user migrator@pve --roles PVEVMAdmin
```

Pre-flight checklist:

- [ ] Target storage has enough free space (disk size + 20% headroom)
- [ ] Management and VM networks are reachable from the operator workstation
- [ ] DNS and NTP work on the target node
- [ ] Backup target (PBS or NFS) is configured and tested
- [ ] A test VM has already been created and booted on the target

4. Proxmox VE Import Assistant for ESXi

Proxmox VE provides an import path that connects to an ESXi host, enumerates VMs, and imports a selected VM's disks and configuration.

Typical UI flow:

1. Datacenter → select target node → **Import VM** (or add an ESXi storage/import source, depending on version)
2. Enter ESXi host address and credentials
3. Select the source VM and target storage
4. Map each source network to a Proxmox bridge/VLAN
5. Choose options (keep MAC address, start after import, etc.)
6. Start import and monitor the task log

```
# Equivalent manual disk import (useful for scripted/air-gapped migrations)
qm create 100 --name migrated-web01 --memory 4096 --cores 2 \
  --net0 virtio,bridge=vmbr0 --scsihw virtio-scsi-single --ostype l26
qm importdisk 100 /mnt/transfer/web01.vmdk local-zfs
qm set 100 --scsi0 local-zfs:vm-100-disk-0 --boot order=scsi0
```

Notes:

- Flatten/consolidate VMware snapshots *before* export or import
- Prefer exporting disks to a neutral format (VMDK or OVF/OVA) when the direct ESXi import is not available
- Keep the original VM's CPU/RAM notes; right-size during import if needed
- Import is I/O heavy — schedule it in a maintenance window and monitor storage latency

5. Migrating Linux Test VMs

Linux guests are usually the easiest: the kernel already contains VirtIO drivers.

Procedure:

1. Power off the source VM (or quiesce it)
2. Import disk and create VM config
3. Set machine type (`q35`), BIOS (SeaBIOS or OVMF), CPU type, disk bus (VirtIO SCSI), NIC (VirtIO)
4. Boot and open the console
5. Verify network, storage, and services
6. Install/enable `qemu-guest-agent`
7. Update `/etc/fstab` if device names/UUIDs changed
8. Test reboot and snapshot

```
# Inside the migrated Linux guest
ip a
lsblk
findmnt
systemctl status qemu-guest-agent || true

# Debian/Ubuntu
apt update
apt install -y qemu-guest-agent
systemctl enable --now qemu-guest-agent

# Fix predictable interface names if needed
# /etc/netplan/00-installer-config.yaml or /etc/network/interfaces
```

Common Linux gotchas:

- Old `/etc/fstab` referencing `/dev/sda1` or stale UUIDs
- NetworkManager connection profiles bound to the old MAC/interface name
- SELinux relabel needed on RHEL/CentOS/Alma/Rocky after major device changes
- Cloud-init metadata pointing at VMware datasources
- VMware Tools left installed (remove or replace with `qemu-guest-agent`)

6. Migrating Windows Server VMs

Windows guests need the most care because the storage controller and NIC change.

Procedure:

1. Prepare the source VM: uninstall VMware Tools (or leave it, then remove later), install VirtIO drivers ISO, ensure latest patches
2. Power off and import
3. Configure VM: `q35`, OVMF (UEFI) if source was UEFI, VirtIO SCSI disk, VirtIO NIC
4. Attach the VirtIO driver ISO (`virtio-win.iso`) as a CD-ROM
5. Boot; if Windows fails with `INACCESSIBLE_BOOT_DEVICE`, boot from ISO and install the VirtIO SCSI driver
6. Install VirtIO NIC driver, QEMU guest agent, and balloon driver
7. Remove VMware Tools, reboot, verify
8. Re-activate Windows if hardware change triggers it (have keys/MAK/KMS ready)

```
# Inside Windows (PowerShell as Administrator)
Get-NetAdapter
Get-Disk
Get-Service QEMU-GA

# Install guest agent from virtio-win ISO
# D:\guest-agent\qemu-ga-x86_64.msi
```

Windows-specific notes:

- Prefer **VirtIO SCSI single** controller with `iothread=1` for busy database VMs
- Keep the same MAC address on import to avoid DHCP/DNS churn where possible
- Windows may re-detect HAL/ACPI; verify time sync and NTP
- Antivirus/EDR agents sometimes bind to VMware adapters — reinstall or update
- Plan for a maintenance window; do not rush driver fixes in production hours

7. Migrating FreeBSD Test VMs

FreeBSD is a good "edge case" test because device naming and drivers differ.

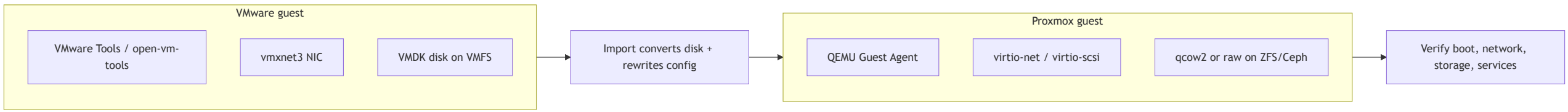
Procedure:

1. Power off source, import disk
2. Configure VM with VirtIO (FreeBSD has `virtio` in the GENERIC kernel)
3. Boot and verify `vtbd0` (disk) and `vtnet0` (NIC)
4. Update `/etc/fstab` if device names changed (e.g. `ada0` → `vtbd0`)
5. Configure networking in `/etc/rc.conf`
6. Install `qemu-guest-agent` from ports/packages and enable it

```
# /etc/rc.conf
ifconfig_vtnet0="DHCP"
qemu_guest_agent_enable="YES"

# /etc/fstab (example)
/dev/vtbd0p2  /  ufs  rw  1  1
```

FreeBSD validates that your **runbook** is not Linux-specific. If a step only works on Linux, document the OS-specific variant.


VMware guest
VMware Tools / open-vm-tools
vmxnet3 NIC
VMDK disk on VMFS
Import converts disk + rewrites config
Proxmox guest
QEMU Guest Agent
virtio-net / virtio-scsi
qcow2 or raw on ZFS/Ceph
Verify boot, network, storage, services

8. Guest Adaptation — Drivers, Boot, Network, Storage

Layer	VMware (before)	Proxmox (after)	Action
Guest agent	open-vm-tools / VMware Tools	QEMU Guest Agent	Replace or remove
NIC	vmxnet3 / e1000	VirtIO net (or e1000 for legacy)	Install driver, fix IP config
Disk controller	LSI Logic / PVSCSI / SATA	VirtIO SCSI / SATA / IDE	Install driver, update fstab

Layer	VMware (before)	Proxmox (after)	Action
Boot firmware	BIOS or UEFI	SeaBIOS or OVMF	Match source firmware type
Time sync	VMware Tools time sync	QEMU guest agent / NTP	Disable double sync conflicts
Power management	VMware ACPI	QEMU ACPI	Verify graceful shutdown works

Boot firmware rule: if the source VM booted with UEFI (and especially Secure Boot), use **OVMF** on Proxmox and import/create the EFI disk. If it used BIOS, use SeaBIOS. Mixing firmware types is a common cause of "no bootable device."

```
# Match firmware and machine type
qm set 100 --bios ovmf --machine q35
qm set 100 --efidisk0 local-zfs:1,efitype=4m,pre-enrolled-keys=1
```

9. Network & Storage Follow-up

Network follow-up checklist:

- [ ] VM NIC is attached to the correct bridge and VLAN tag
- [ ] MAC address policy decided (keep vs. regenerate) and documented
- [ ] IP configuration (static/DHCP), DNS, NTP verified
- [ ] MTU matches (1500 default; 9000 only if end-to-end jumbo frames)
- [ ] Firewall rules on datacenter/node/VM levels reviewed
- [ ] Outbound connectivity and name resolution tested from the guest

```
# Proxmox side
qm config 100 | grep net0
qm set 100 --net0 virtio,bridge=vmbr0,tag=20,firewall=1
```

Storage follow-up checklist:

- [ ] Disk bus is VirtIO SCSI (or SATA/IDE for legacy guests)
- [ ] `scsihw` is `virtio-scsi-single` (or `virtio-scsi-pci`)
- [ ] Cache mode chosen (`none` for data safety, `writeback` for performance with battery-backed storage)
- [ ] `iothread=1` and `discard=on` where appropriate
- [ ] TRIM/discard tested inside the guest
- [ ] Snapshot and backup tested on the migrated VM

```
qm set 100 --scsi0 local-zfs:vm-100-disk-0,iothread=1,discard=on,cache=none
qm config 100
```

10. Function Tests for Migrated VMs

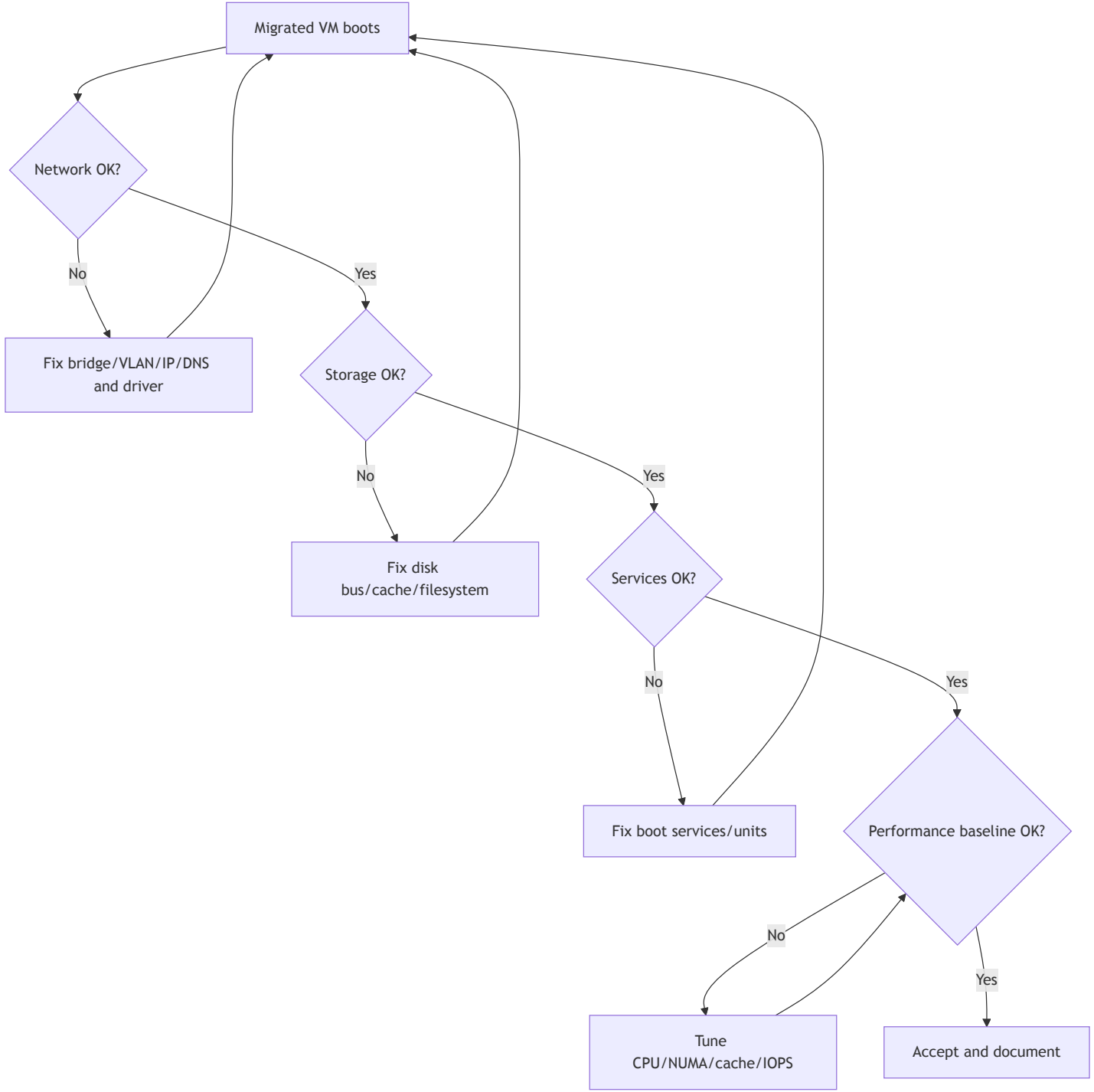
A migrated VM is not "done" when it boots. It is done when it passes a written acceptance test.

Test	How	Pass criteria
Boot	Start VM, watch console	Reaches login prompt without errors
Reboot	Guest reboot	Returns to running state automatically
Shutdown	`qm shutdown`	Clean ACPI shutdown, no forced stop
Network L2/L3	Ping gateway, DNS lookup	All pass

Test	How	Pass criteria
Services	`systemctl` / Services.msc	All expected services running
Application	App-specific smoke test	Login, CRUD, report generation
Storage	Read/write test file, check mounts	No errors, expected capacity
Backup	Run backup job	Completes successfully
Snapshot	Create + revert snapshot	Reverts cleanly
Agent	`qm guest cmd` / guest exec	Agent responds

```
# Proxmox-side smoke tests
qm status 100
qm guest exec 100 -- ip a
qm snapshot 100 post-migration-test
qm rollback 100 post-migration-test
```

Document results per VM in the migration tracker (spreadsheet or ticket). This becomes your audit trail.


Migrated VM boots
Network OK?
No
Yes
Fix bridge/VLAN/IP/DNS
and driver
Storage OK?
No
Yes
Fix disk
bus/cache/filesystem
Services OK?
No
Yes
Fix boot services/units
Performance baseline OK?
No
Yes
Tune
CPU/NUMA/cache/IOPS
Accept and document

11. Troubleshooting Import & Boot Issues

Symptom	Likely cause	Fix
Import fails / timeout	Network, credentials, storage full, large disk	Check task log, free space, retry, split disks
"No bootable device"	Wrong firmware (BIOS vs UEFI) or boot order	Match firmware; set `--boot order=scsi0`
Kernel panic / initramfs	Missing disk driver or stale fstab	Boot rescue ISO, rebuild initramfs, fix fstab
Windows `INACCESSIBLE_BOOT_DEVICE`	Missing VirtIO SCSI driver	Attach virtio-win ISO, install driver
No network after boot	Driver not installed or wrong bridge/VLAN	Install driver; verify `qm config` net0

Symptom	Likely cause	Fix
DHCP gets wrong/no lease	MAC changed or VLAN mismatch	Keep MAC or update DHCP reservation
Disk read-only / fs errors	Cache mode, controller mismatch	Set correct bus/cache; run fsck
Time drift	Double time sync or no NTP	Disable VMware sync, enable NTP
Guest agent not responding	Agent not installed/enabled	Install and start qemu-guest-agent
Performance regression	IDE/SATA bus, no iothread, ZFS misconfig	Switch to VirtIO SCSI; tune storage

General troubleshooting discipline:

1. Read the task log in the Proxmox UI first
2. Check `qm config <vmid>` — does it match your intent?
3. Check guest console, not just SSH
4. Change one thing at a time and re-test
5. Keep the source VM untouched for rollback

12. Performance Baseline & Tuning

Establish a **baseline on VMware before migration** and compare after. Without a baseline, "it feels slower" cannot be resolved.

Metrics to capture (both sides):

Metric	Tool
CPU usage / ready time	`esxtop`, `mpstat`, `top`
Memory usage / ballooning	`esxtop`, `free`, `vmstat`
Disk IOPS / latency / throughput	`iostat`, `fio`, vendor tools
Network throughput / latency	`iperf3`, `ping`, `ss`
Application response time	App metrics / APM

```
# Quick disk benchmark inside a guest (use with care in production)
fio --name=randread --ioengine=libaio --iodepth=32 \
  --rw=randread --bs=4k --direct=1 --size=1G --runtime=60

# Network
iperf3 -c 10.10.10.1 -t 30
```

Tuning levers on Proxmox:

- CPU type: `host` for max performance (breaks cross-vendor live migration) vs. named model for portability
- NUMA: enable `numa: 1` on large VMs on multi-socket hosts
- Disk: VirtIO SCSI + `iothread=1`, appropriate cache, `discard=on`
- Network: VirtIO multiqueue (`queues=4`) for high-throughput VMs
- Storage: ZFS recordsize/ARC, Ceph pool tuning, SSD vs HDD placement
- Hugepages for latency-sensitive or large-memory workloads

13. Lab: Migrate and Validate Three VMs

Lab goals:

1. Migrate one **Linux** VM, install QEMU guest agent, verify services and reboot
2. Migrate one **Windows Server** VM, install VirtIO drivers and guest agent, verify
3. Migrate one **FreeBSD** (or alternate Linux) VM to test OS-specific adaptation
4. Run the function-test matrix on each and record results
5. Create and revert a snapshot on one migrated VM
6. Run a simple performance comparison against your baseline notes

Deliverable: a completed migration tracker row per VM (config, issues found, fixes applied, test results, go/no-go).

14. Import Runbook — Step-by-Step

A runbook makes imports repeatable and auditable.

Pre-import:

1. Confirm source VM powered off
2. Confirm snapshot consolidation complete
3. Verify target storage free space
4. Verify network mapping documented
5. Notify application owner

Import:

6. Create VM shell with correct firmware/machine type
7. Import disk(s)
8. Attach disk(s) and set boot order
9. Map NICs to bridges/VLANs
10. Set CPU/RAM to planned values

Post-import:

11. Boot and open console
12. Verify guest OS network/storage
13. Install/enable QEMU guest agent
14. Run function tests
15. Create baseline snapshot
16. Record results in tracker

14b. Import Runbook — Validation Gates

Gate	Check	Owner
G1	Source VM off and snapshot-free	VMware SME
G2	Target capacity ≥ required + 20%	Storage
G3	Network mapping approved	Network
G4	Import task completed without errors	Migration lead

Gate	Check	Owner
G5	Guest boots to login	Migration lead
G6	Network L3 + DNS works	Migration lead
G7	Application smoke test passes	App owner
G8	Backup + snapshot tested	Migration lead

No VM advances to production without all gates green.

15. Linux Migration Deep Dive

Pre-migration (source):

- Update OS and kernel
- Remove stale VMware Tools if not needed
- Document network config and routes
- Document fstab and mount points
- Capture `ip a`, `lsblk`, `systemctl --failed`

Post-migration (target):

- Install `qemu-guest-agent`
- Fix interface names (netplan/networkd/NetworkManager)
- Rebuild initramfs if disk driver changed
- Verify SELinux labels (RHEL family)
- Remove VMware-specific services
- Reboot twice and compare

Linux Network Fix Examples

Debian/Ubuntu (netplan):

```
network:
  version: 2
  ethernets:
    ens18:
      dhcp4: false
      addresses: [10.20.0.10/24]
      routes:
        - to: default
          via: 10.20.0.1
      nameservers:
        addresses: [10.10.10.53]
```

RHEL/Alma (NetworkManager):

```
nmcli con mod eth0 ipv4.addresses 10.20.0.10/24
nmcli con mod eth0 ipv4.gateway 10.20.0.1
nmcli con mod eth0 ipv4.dns 10.10.10.53
nmcli con up eth0
```

Linux Storage Fix Examples

```
# Find new disk UUIDs
blkid
lsblk -o NAME,UUID,FSTYPE,SIZE

# Update /etc/fstab using UUIDs
# UUID=xxxx / ext4 defaults 0 1

# Rebuild initramfs (Debian/Ubuntu)
update-initramfs -u -k all

# RHEL family
dracut -f -v

# Test TRIM
fstrim -av
```

SELinux Relabel (RHEL Family)

```
# If boot fails or services denied after disk/controller change
touch /.autorelabel
reboot
# Or from rescue:
mount /dev/vtbd0p2 /mnt
touch /mnt/.autorelabel
```

Check denials:

```
ausearch -m avc -ts recent
```

16. Windows Migration Deep Dive

Pre-migration:

- Install latest Windows updates
- Snapshot/backup
- Download VirtIO ISO (`virtio-win.iso`)
- Note installed VMware Tools version
- Document services and scheduled tasks

Import config:

- Machine: `q35`
- BIOS: OVMF for UEFI sources
- Disk: VirtIO SCSI (or SATA for first boot)
- NIC: VirtIO (or e1000 for compatibility)

Post-migration:

- Install VirtIO drivers and guest agent
- Remove VMware Tools
- Verify activation
- Reboot and test

Windows Driver Installation Order

1. Boot with SATA/IDE if VirtIO SCSI driver missing
2. Install `vioscsi` (storage) from virtio-win ISO
3. Switch disk to VirtIO SCSI and reboot
4. Install `netkvm` (network) driver
5. Install balloon and qemupciserial drivers
6. Install QEMU guest agent
7. Remove VMware Tools
8. Reboot and verify Device Manager (no unknown devices)

Windows Activation & Licensing

- Hardware change may trigger reactivation
- Have MAK/KMS keys and KMS server details ready
- Document activation status before and after
- For volume-licensed VMs, ensure KMS reachability from new network
- Test with `slmgr /dlv` and `/ato`

```
slmgr /dlv
slmgr /ato
```

Windows Performance Tuning

- Use VirtIO SCSI single controller with `iothread=1`
- Enable multiqueue for high-throughput VMs
- Set CPU type appropriately (`host` vs. named model)
- Disable unnecessary services (print spooler, etc.) if server role
- Keep pagefile on fast storage
- Verify TRIM/discard if supported

```
# Check disk alignment and fragmentation
Get-PhysicalDisk | Select FriendlyName, MediaType, Size
Optimize-Volume -DriveLetter C -ReTrim -Verbose
```

17. Application-Tier Migration Patterns

Tier	Considerations	Test
Database	Consistency, I/O latency, backup	Query perf, integrity check
App server	Sessions, config paths, service accounts	Login, transactions
Web frontend	TLS certs, DNS, load balancer	HTTP smoke, SSL scan
Identity (AD)	Replication, FSMO, DNS	Auth, group policy
File server	Shares, ACLs, quotas	Access, throughput
Monitoring	Agents, endpoints, dashboards	Alerts, graphs

Migrate dependencies together or use maintenance windows.

Database Migration Notes

- Prefer application-level backup/restore for consistency (pg_dump, mysqldump, RMAN)
- If cold-importing, ensure DB is shut down cleanly
- Test restore on target before cutover
- Compare row counts and checksums
- Rebuild indexes if storage layout changed
- Monitor slow queries after migration

```
# PostgreSQL example
pg_dump -Fc -f /backup/db.dump dbname
pg_restore -d dbname /backup/db.dump
```

18. Data Migration Strategies

Strategy	Downtime	Best for
Cold VMDK import	Hours	Most VMs
OVF/OVA export+import	Hours	Portability, vendor appliances
Shared storage swing	Minutes	When source and target share storage
Replication + cutover	Minutes	Large VMs, DBs
Application-level	Variable	DBs, clustered apps

Choose per workload; mix strategies in one project.

OVF/OVA Flow

```
# Export from vSphere (ovftool)
ovftool vi://user@vcenter/vm/web01 /transfer/web01.ova

# Import into Proxmox
qm importovf 100 /transfer/web01.ova local-zfs
```

- OVA includes config + disks
- Good for vendor appliances
- Verify CPU/RAM and network after import

19. Automation of Migration

Scripted import example:

```
#!/bin/bash
set -euo pipefail
VMID=$1; NAME=$2; DISK=$3; STORAGE=$4
qm create $VMID --name "$NAME" --memory 4096 --cores 2 \
  --net0 virtio,bridge=vmbr0 --scsihw virtio-scsi-single --ostype l26
qm importdisk $VMID "$DISK" "$STORAGE"
qm set $VMID --scsi0 "$STORAGE:vm-${VMID}-disk-0" --boot order=scsi0
qm set $VMID --agent enabled=1
```

- Store scripts in Git
- Use Ansible/Terraform for repeatable target config
- Log every run

Terraform for Proxmox (Example)

```
resource "proxmox_vm_qemu" "web01" {
  name        = "web01"
  target_node = "pve1"
  cores       = 2
  memory      = 4096
  clone       = "ubuntu-template"
  network {
    model  = "virtio"
    bridge = "vmbr0"
  }
}
```

Use IaC for new VMs; for migrated VMs, import then manage config.

20. Validation Framework

Validation layers:

1. Infrastructure (boot, network, storage)
2. OS (services, time, users)
3. Application (smoke tests, transactions)
4. Performance (baseline comparison)
5. Security (firewall, agents, patches)
6. Backup/DR (backup + restore test)

Each layer has pass/fail criteria and an owner.

Test Data & Environments

- Use synthetic test data where possible
- Mask production data for dev/test
- Keep a golden dataset for regression
- Automate smoke tests (curl, SQL queries, Selenium)
- Store test scripts in Git alongside runbooks

21. Performance Tuning Deep Dive

CPU:

- `host` for max performance; named model for portability
- NUMA alignment for large VMs
- Avoid overcommit on latency-sensitive workloads

Memory:

- Disable ballooning for DB VMs if needed
- Use hugepages for large memory
- Monitor swap usage

Disk:

- VirtIO SCSI single + iothread
- Cache mode per workload
- ZFS recordsize/ARC tuning

Network:

- VirtIO multiqueue
- MTU consistency
- Offloads enabled

Performance Test Script (Example)

```
#!/bin/bash
# CPU
sysbench cpu --cpu-max-prime=20000 run
# Memory
sysbench memory run
# Disk
fio --name=randrw --ioengine=libaio --iodepth=32 --rw=randrw --bs=4k --direct=1 --size=2G --runtime=120 --group_reporting
# Network
iperf3 -c 10.20.0.1 -t 60 -P 4
```

Record results per VM and compare to baseline.

22. Troubleshooting Deep Dive

Boot issues:

- Check firmware type (BIOS vs UEFI)
- Check boot order (`qm config`)
- Inspect VM console (not just SSH)
- Use rescue ISO to chroot and fix

Network issues:

- Verify bridge/VLAN on host
- Verify guest driver and IP config
- Test from host: `tcpdump` on bridge

Storage issues:

- Check disk attachment and bus
- Verify filesystem health
- Check ZFS/Ceph status

Rescue Workflow Example

```
# Boot VM from a rescue ISO (attach ISO, set boot order)
qm set 100 --ide2 local:iso/systemrescue.iso
qm set 100 --boot order=ide2

# In rescue shell:
lsblk
mount /dev/vtbd0p2 /mnt
# Fix fstab, reinstall grub, rebuild initramfs
grub-install /dev/vtbd0
update-initramfs -u
```

23. Security During Migration

- Do not disable firewalls "temporarily" without a ticket and expiry
- Keep migration network isolated and remove after cutover
- Rotate any credentials used during import
- Scan migrated VMs for malware/old agents
- Remove VMware Tools where not needed
- Patch guest OS before and after migration
- Verify backup encryption keys

24. Documentation & Handover

Handover package:

- Final inventory and wave results
- Architecture diagrams (network/storage)
- Runbooks (import, rollback, backup restore)
- Monitoring dashboards and alerts
- Credential and token inventory (in vault)
- Known issues and technical debt
- Training plan for operations team

Schedule a handover workshop and a 30-day follow-up review.

25. Appendix A — Import Troubleshooting Matrix

Symptom	Cause	Fix
Import hangs	Network/storage slow	Check logs, retry off-peak
Disk not found	Wrong storage path	Verify `pvesm status`
Boot loop	Firmware mismatch	Match BIOS/UEFI
No network	Driver/bridge/VLAN	Install driver, fix config
Slow I/O	Cache/bus misconfig	Tune VirtIO + storage
Agent absent	Not installed/enabled	Install + enable service

Appendix B — Linux Quick Reference

```
# Guest agent
systemctl status qemu-guest-agent
# Network
ip a; ip r; resolvectl status
# Storage
lsblk; findmnt; df -h
# Services
systemctl --failed
# Logs
journalctl -xe --since "1 hour ago"
```

Appendix C — Windows Quick Reference

```
Get-NetAdapter
Get-Disk
Get-Service QEMU-GA
slmgr /dlv
Get-WindowsFeature  # Server roles
Test-NetConnection 8.8.8.8
```

Appendix D — Performance Baseline Template

VM	CPU score	Mem (MB/s)	Disk IOPS	Disk lat (ms)	Net (Gbps)	Notes
web01						
db01						

Fill before and after migration; compare deltas.

26. Import Verification Scripts

Linux verification script:

```
#!/bin/bash
set -e
echo "Hostname: $(hostname)"
ip -brief a
echo "Default route:"; ip route | grep default
echo "DNS:"; getent hosts example.com
echo "Disk:"; lsblk
echo "Agent:"; systemctl is-active qemu-guest-agent
echo "Failed units:"; systemctl --failed --no-legend
```

Run after every import and save output to tracker.

27. Automated Testing with Ansible

```
- name: Validate migrated VM
  hosts: migrated
  tasks:
    - name: Check SSH
      ansible.builtin.wait_for:
        port: 22
        timeout: 30
    - name: Check service
      ansible.builtin.service:
        name: nginx
        state: started
```

Automate smoke tests across waves.

28. Rollback Drills

Lab drill:

1. Import a test VM
2. Simulate failure (wrong network)
3. Execute rollback (power on source, revert DNS)
4. Measure rollback time
5. Document improvements

Practice rollback until it is boring.

29. Data Integrity Checks

- Compare file counts and sizes
- Database row counts and checksums
- Application-level validation
- Hash critical files before/after
- Verify backups restore correctly

```
# Example: checksum a directory
find /data -type f -exec sha256sum {} \; | sort > /tmp/before.txt
```

30. Performance Troubleshooting

Symptom	Check	Fix
High CPU ready	Overcommit	Reduce vCPU or add host
High I/O latency	Storage contention	Tune cache/recordsize
Network drops	MTU mismatch	Align MTU
Memory pressure	Ballooning	Add RAM or reduce guests
Slow boot	Disk bus	Switch to VirtIO

31. OS-Specific Notes: Ubuntu/Debian

- Use netplan for network config
- `cloud-init` may need datasource change
- Install `qemu-guest-agent`
- Update initramfs after disk changes
- AppArmor profiles may need tuning

32. OS-Specific Notes: RHEL/Alma/Rocky

- NetworkManager profiles
- SELinux relabel after major changes
- `dracut` for initramfs
- Firewall (firewalld) rules
- Subscription manager (if applicable)

33. OS-Specific Notes: SUSE

- wicked network config
- YaST for network/storage
- AppArmor
- `mkinitrd` for initramfs
- SUSE-specific agents

34. OS-Specific Notes: Windows Server

- VirtIO drivers from virtio-win
- QEMU guest agent
- Activation and KMS
- Windows Firewall rules
- Antivirus/EDR reconfiguration
- Pagefile placement

35. Migration of Virtual Appliances

- Often OVF/OVA-based
- May have hardcoded MAC/IP assumptions
- Check vendor support for Proxmox/KVM
- May require specific firmware (BIOS/UEFI)
- Test upgrades separately

36. Guest OS Licensing

- Windows: activation may trigger; have keys ready
- RHEL: subscription may need re-registration
- SUSE: registration codes
- Oracle: license compliance on new hardware
- Document license changes in migration tracker

37. Post-Migration Optimization

- Right-size vCPU/RAM based on actual usage
- Move hot disks to faster storage
- Enable multiqueue for busy NICs
- Tune ZFS/Ceph
- Remove unused snapshots
- Update monitoring thresholds

38. Containerization Decision (Later)

After stable VM migration, evaluate:

- Which Linux VMs could become containers?
- App compatibility with LXC
- Security boundary requirements
- Operational skill set
- Do as a separate project with its own testing

39. Migration of Monitoring Stack

- Migrate monitoring server early (as a VM) or keep on VMware until end
- Update agent endpoints to new IPs
- Test alerts and dashboards
- Monitor both old and new during transition

40. Migration of Backup Infrastructure

- PBS can run as VM initially
- Migrate backup jobs gradually
- Test restore from PBS to Proxmox VMs
- Keep old backups until retention expires
- Document backup/restore runbooks

41. Final Acceptance Criteria

A migration project is complete when:

- All in-scope VMs migrated and accepted
- Performance within agreed thresholds
- Backups and DR tested
- Monitoring and alerting live
- Documentation handed over
- Operations team trained
- Retrospective completed
- Source environment decommissioned per policy

42. Appendix: Extended Command Reference

```
# Migration helpers
qm importdisk <vmid> <file> <storage>
qm importovf <vmid> <ovf> <storage>
qm set <vmid> --boot order=scsi0
qm template <vmid>
qm clone <vmid> <newid> --full

# Cluster
pvecm status
pvecm add <host>

# Storage
pvesm status
pvesm add zfspool <id> --pool <pool>
```

43. Appendix: Troubleshooting Flowchart

1. Check task log
2. Check `qm config`
3. Check guest console
4. Check host logs
5. Check network/storage health
6. Rollback if needed

44. Appendix: Lab Checklist (Detailed)

- [ ] 3 VMs selected
- [ ] Baselines recorded
- [ ] Source powered off
- [ ] Imports completed
- [ ] Guests adapted
- [ ] Function tests passed
- [ ] Performance compared
- [ ] Snapshots tested
- [ ] Backups tested
- [ ] Results documented

45. Appendix: Q&A Prompts

- Which VM would you migrate first and why?
- What is your rollback trigger?
- How do you verify a migrated VM?
- What performance metrics matter most?
- How do you handle Windows activation?

Day 2 — Key Takeaways

- ESXi import is a **cold migration** — plan downtime and consistency
- Linux guests are usually easy; **Windows guests** need VirtIO driver handling
- **Guest adaptation** (agent, NIC, disk, firmware, time sync) is where time is spent
- A written **function-test matrix** defines "done," not "it booted"
- Troubleshoot from the **task log and** `qm config` before guessing
- Always keep the **source VM untouched** until acceptance criteria pass